

COMPX341-23A

Advanced Software Engineering Design

Week 7
Lecture 17

Unit testing example

“Create a class `IntSet` which provides set operations for sets of integers. You should be able to add to a set, test whether an element is in a set, delete from a set, form the intersection and unions of sets.”

Unit testing example

Look at implementation and a “test” run with printing to “debug”...

Unit testing example

- Alternatively, write a test suite...
- Units:
 - Write unit tests to test the method that adds elements to a set
 - Write unit tests to test the method that deletes elements from a set
 - Write unit tests to test the method that forms intersections of sets
 - Write unit tests to test the method that forms unions of sets

Unit testing example

```
// Arrange  
IntSet a = new IntSet(5)  
  
// Act  
a.add(1); a.add(2); a.add(3);  
  
// Assert  
a.hasElement(4)
```

Unit testing example

```
// Arrange  
IntSet a = new IntSet(5)  
  
// Act  
a.add(1); a.add(2); a.add(3);  
  
// Assert  
a.hasElement(3)
```

Unit testing example

```
// Arrange  
IntSet a = new IntSet(5)  
IntSet b = new IntSet(5)  
  
// Act  
a.add(1) a.add(2) a.add(3) b.add(3)  
IntSet c = a.intersection(b)  
  
// Assert  
c.hasElement(3)
```

JUnit

JUnit

- A very common package for writing, organising, and running tests in Java
- You will need to set up your environment for JUnit (sometimes)
- I'm using JUnit4 since it's simpler than the latest version but does everything we need
- Just use an editor and a terminal (IDEs can be unwieldy and obscure the simple ideas)
- Good introduction, which I'll work through (and add to):

<https://github.com/junit-team/junit4/wiki/Getting-started>

Writing tests with JUnit

A JUnit test method has three requirements:

1. It isn't static or private
2. It can't return anything (must be void)
3. A test method must be annotated with `@Test`

Writing tests with JUnit

In JUnit, annotations are used to specify when methods should run during testing.

Some other common annotations include:

- **@Before**: execute this method before each test
- **@After**: execute this method after each test

Writing tests with JUnit

In JUnit, values are evaluated using “assertions”.

Some common assertions include:

- **assertEquals(x, y)**: asserts that x and y are equal
- **assertNotEquals(x, y)**: asserts that x and y are not equal
- **assertTrue(boolean condition)**: asserts that the supplied condition is true
- **assertFalse(boolean condition)**: asserts that the supplied condition is not true
- **assertNotNull(Object actual)**: asserts that actual is not null
- ... and lots more!

<https://github.com/junit-team/junit4/wiki/Assertions>

For example ...

Assume that we have the following class:

```
public class HelloWorld {  
  
    public String helloWorld(){  
        return "Hello World!";  
    }  
}
```

We want to test the “helloWorld” method to determine if it returns the correct string.

Example Test

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class HelloWorldTest {
    @Test
    public void helloWorldTest() {
        HelloWorld t = new HelloWorld();
        assertEquals(t.helloWorld(), "Hello World!");
    }
}
```

Examples.....see the files attached to this week on Moodle

TestJUnit

Calculator

IntSet, TestTrivIntSet and TestIntSet